ПРАКТИЧЕСКАЯ РАБОТА · МДК.09.01

№3. Подключение Flask и первый маршрут

Выполнила: Миронова Анна Андреевна
Группа: 4-ИСП9-46

Преподаватель: А. Кондыбаева
Дата выполнения: 08.11.2026
Репозиторий: https://github.com/RealLalka/mdk-coursework-mironova-2026

Цель работы

Освоить: подключение flask и первый маршрут.

Условие: практическая работа №3.

Реализованный функционал

Создан app.py с маршрутами / и /about, взаимными ссылками и обработкой неизвестного URL штатным 404. Сервер привязан к loopback. Режим отладки включается только явной переменной FLASK_DEBUG=1.

Запуск

```
python web/practice_03/app.py
```

Стенд и воспроизведение

Фактическая ОС: Arch Linux в WSL/Linux, ядро 6.6.87.2-microsoft-standard-WSL2. Python 3.14.6, SQLite 3.53.3, Flask 3.1.2; браузер Chromium; настольный Postman 9.31.30 и Newman 6.2.1 для API-прогонов. Условия некоторых заданий называют РОСА и Python 3.8: переносимые команды выполнены на текущем Linux; совместимость именно с РОСА и Python 3.8 отдельным прогоном не подтверждена. Ограничения окружения отражены как WARN/SKIP, а не как успешные проверки.

Команды установки и запуска приведены в README. Полный автоматический прогон: `python tools/verify.py --workspace <новый-каталог-результатов> --newman <путь-к-newman>`. Он создаёт изолированные копии, запускает локальные сервисы и отрицательные сценарии, затем останавливает собственные процессы. Ненулевой код ожидается только в явно помеченных отрицательных опытах.

Анализ сложности

Маршруты возвращают фиксированный HTML: O(1) относительно числа задач. Для ответа длиной L затраты записи ответа O(L), память O(L). База данных в этой работе не используется.

Скриншоты и примеры выполнения



Рисунок 1. web03-home. Фактический запуск; для консольных работ показан оформленный протокол вывода, для веб-приложений и Postman — интерфейс программы.



Рисунок 2. web03-about. Фактический запуск; для консольных работ показан оформленный протокол вывода, для веб-приложений и Postman — интерфейс программы.

Ответы на контрольные вопросы

1. Что делает @app.route?

Регистрирует соответствие пути URL и функции обработчика. Для / вызывается index.

2. GET и POST; метод по умолчанию?

GET получает представление ресурса; POST отправляет данные для обработки. Flask регистрирует GET по умолчанию, также автоматически поддерживает HEAD и OPTIONS.

3. Зачем debug=True и почему не в production?

Отладка включает подробную диагностику и обычно автоперезагрузку при изменении файлов. Интерактивный debugger раскрывает внутреннее состояние и может выполнять код; вне локального стенда его включать нельзя.

4. Что означает 127.0.0.1:5000?

127.0.0.1 — loopback текущего компьютера; 5000 — TCP-порт. С другого компьютера этот адрес ведёт к нему самому, а не к нашему серверу.

5. Почему неизвестный URL даёт 404?

Маршрутизатор не находит соответствующий обработчик и формирует HTTP-ответ Not Found. Это штатная обработка запроса.

6. Почему templates и можно ли изменить имя?

Это стандартный каталог Flask для Jinja2. Имя можно изменить параметром template_folder. В практике №3 HTML возвращается строкой, поэтому этот каталог ещё не требуется.

7. Два сервера на одном порту?

Если оба пытаются слушать один адрес и порт, второй обычно завершится с Address already in use. Нужно остановить первый процесс либо выбрать другой PORT.

Протоколы проверок

additional_checks

Полный протокол: evidence/additional_checks.md.

```
implicit: table persists without commit = True
explicit: table persists without commit = False
Flask debug reloader: Hello, Flask! -> Hello, Reload! PASS; isolated copy only
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
 * Running on http://127.0.0.1:33669
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
127.0.0.1 - - [08/Nov/2026 19:45:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Nov/2026 19:45:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Nov/2026 19:45:25] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Nov/2026 19:45:26] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Nov/2026 19:45:26] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Nov/2026 19:45:26] "GET / HTTP/1.1" 200 -
```

compile

Полный протокол: evidence/compile.md.

Вывод

Требования выбранной практики реализованы. Результат проверен фактическим выполнением и автоматическими проверками; детали и границы проверки приведены выше.

Использованные материалы

Условия работ: репозитории softboxdev/programming_course и softboxdev/test_methodologies; точные ссылки и commit SHA зафиксированы в SOURCES.md.

Документация: Python sqlite3, Python json, Flask Quickstart, Postman Newman.

Приложение. Листинги файлов

web/practice_03/app.py

```
import os
from flask import Flask

app = Flask(__name__)


@app.route("/")
def index():
    return '<h1>Hello, Flask!</h1><p>Первый маршрут работает.</p><a href="/about">О проекте</a>'


@app.route("/about")
def about():
    return '<h1>О проекте</h1><p>Учебное приложение на Flask. Практика №3.</p><a
href="/">Главная</a>'


if __name__ == "__main__":
    app.run(
        host="127.0.0.1",
        port=int(os.environ.get("PORT", "5000")),
        debug=os.environ.get("FLASK_DEBUG") == "1",
    )
```